

Introduction à R

Bioinformatique I (BIF-7900)

Antoine Bodein

1er octobre 2026

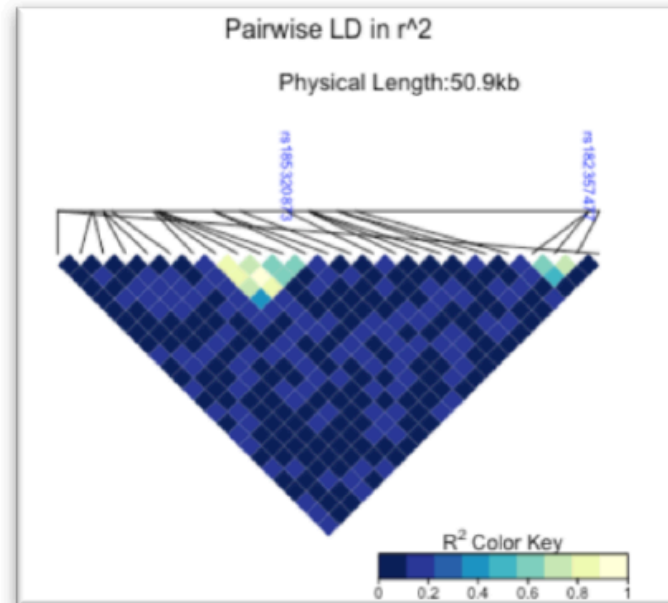
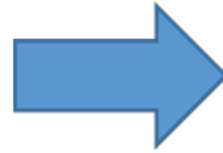


| 1

INTRODUCTION

Objectifs

Utilisation *minimale* de R dans un contexte de recherche scientifique :



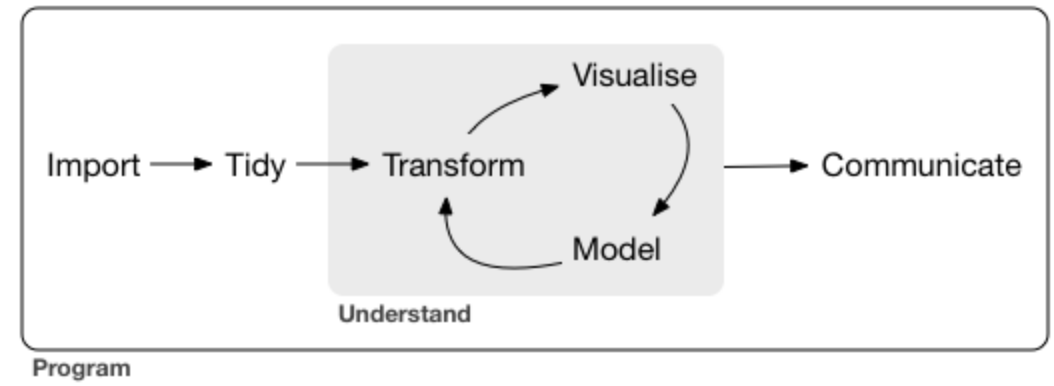
1. Pouvoir modifier des données
2. Pouvoir extraire des sous-ensembles de données (*indexing*)
3. Pouvoir découvrir et utiliser une fonction
4. Produire des scripts pour un travail reproductible et documenté

Plan

1. Introduction
2. Environnement de travail
3. Un premier contact avec R : variables, vecteurs, types, matrices
4. Les **data.frame** → **TP1**
5. Fonctions, aide et packages
6. Les boucles
7. Importer et exporter des données
8. Les scripts → **TP2** et **TP3**
9. Activité : auditer un script généré par IA

Présentation du langage R

- Un langage **interprété** (comme Bash ou Python, par opposition aux langages compilés comme C++ ou Java)
- Spécialisé pour les analyses statistiques et la visualisation de données



Une suite logicielle, ou environnement intégré, qui comprend :

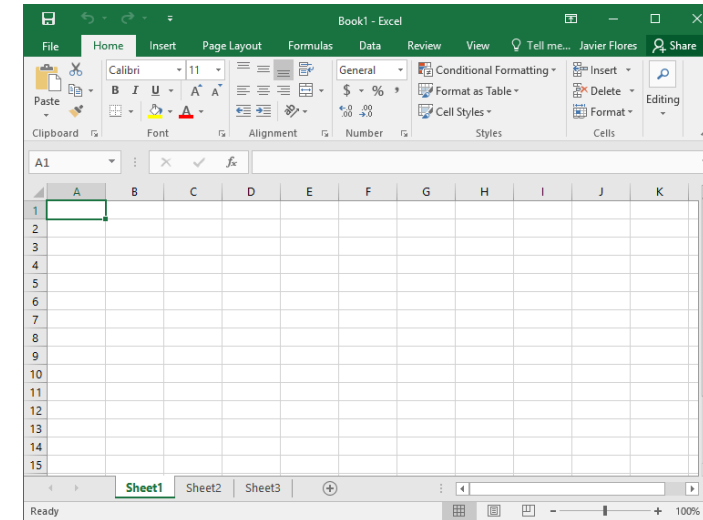
- un langage de programmation
- un programme interactif (REPL : *read-eval-print loop*)
- des outils graphiques
- un système d'extensions (*packages*)

R for Data Science (H. Wickham)

Pourquoi apprendre R

- Analyse de données (*data science*)
- Données diverses (biostatistiques, génomiques, biologiques)
- Graphiques de haute qualité (articles scientifiques)
- Grande communauté active qui ajoute des fonctionnalités et permet des analyses spécialisées

Quand utiliser R : **explorer**, **analyser**, **représenter**, **communiquer** les données.



Ceci n'est pas R !

Pourquoi apprendre R en 2026 ?

Un assistant d'IA écrit du code R en quelques secondes. Alors pourquoi apprendre ?

L'assistant sait...	Vous devez savoir...
écrire du code plausible	vérifier qu'il fait ce que vous vouliez
produire un code qui s'exécute sans erreur	repérer les erreurs qui ne plantent pas
proposer une analyse	en assumer la responsabilité : c'est vous qui signez
travailler sur des données qu'on lui envoie	travailler quand les données ne peuvent pas sortir (données cliniques)

Objectif du module : comprendre, vérifier et corriger une analyse, la vôtre ou celle d'un outil. C'est ce qui est évalué, en présence et sans IA.

Pourquoi R plutôt que Python ?

R : le choix du cours

Bioconductor : plus de 2 000 packages omiques (DESeq2, limma, mixOmics...)

Statistiques natives : `data.frame`, `NA`, tests, modèles

Vectorisé par défaut, RStudio prêt à l'emploi

Visualisation, statistiques, réseaux et enrichissement **de ce cours** sont en R

Python : à apprendre ensuite

Pipelines (Snakemake), automatisation

Apprentissage automatique (scikit-learn, PyTorch)

Single-cell (scanpy), structures, Biopython

Langage **généraliste**, très demandé hors recherche

Les concepts se transfèrent : vecteur ↔ tableau `numpy`, `data.frame` ↔ `DataFrame` pandas, `NA` ↔ `NaN`.
Maîtriser l'un rend l'autre facile.

R : mode interactif vs. mode script

Le mode **interactif** : utile pour explorer les données (*Ctrl+D pour quitter*)

```
[→ ~ R

R version 4.1.0 (2021-05-18) -- "Camp Pontanezen"
Copyright (C) 2021 The R Foundation for Statistical Computing
Platform: aarch64-apple-darwin20 (64-bit)

R est un logiciel libre livré sans AUCUNE GARANTIE.
Vous pouvez le redistribuer sous certaines conditions.
Tapez 'license()' ou 'licence()' pour plus de détails.

R est un projet collaboratif avec de nombreux contributeurs.
Tapez 'contributors()' pour plus d'information et
'citation()' pour la façon de le citer dans les publications.

Tapez 'demo()' pour des démonstrations, 'help()' pour l'aide
en ligne ou 'help.start()' pour obtenir l'aide au format HTML.
Tapez 'q()' pour quitter R.

[> 1+1
[1] 2
> █
```

Le mode **script** (**Rscript**) : utile pour réutiliser des commandes utilisées régulièrement. Conserve aussi le détail d'une analyse (*reproductibilité*)

```
[→ ~ vi my_script.R
[→ ~ Rscript my_script.R
[1] 2
→ ~ █
```

L'interpréteur

- À la console, l'interpréteur attend qu'on communique avec lui dans le langage qu'il comprend : **R**.
- Toute erreur de syntaxe entraîne une incompréhension de l'interpréteur et produit une erreur.

Erreurs fréquentes :

- mauvaise orthographe (R distingue majuscules et minuscules) ;
- utiliser une fonction inconnue ;
- oublier de fermer un guillemet ou une parenthèse ;
- argument manquant à une fonction (pas de valeur par défaut).

| 2

ENVIRONNEMENT DE TRAVAIL

Commencer avec R

- R peut être téléchargé et installé à partir de <https://cran.r-project.org/>
- Pour le cours, un serveur **RStudio** a été installé pour vous : <https://bif-rstudio.compbio.ulaval.ca>

Sign in to RStudio

Username:

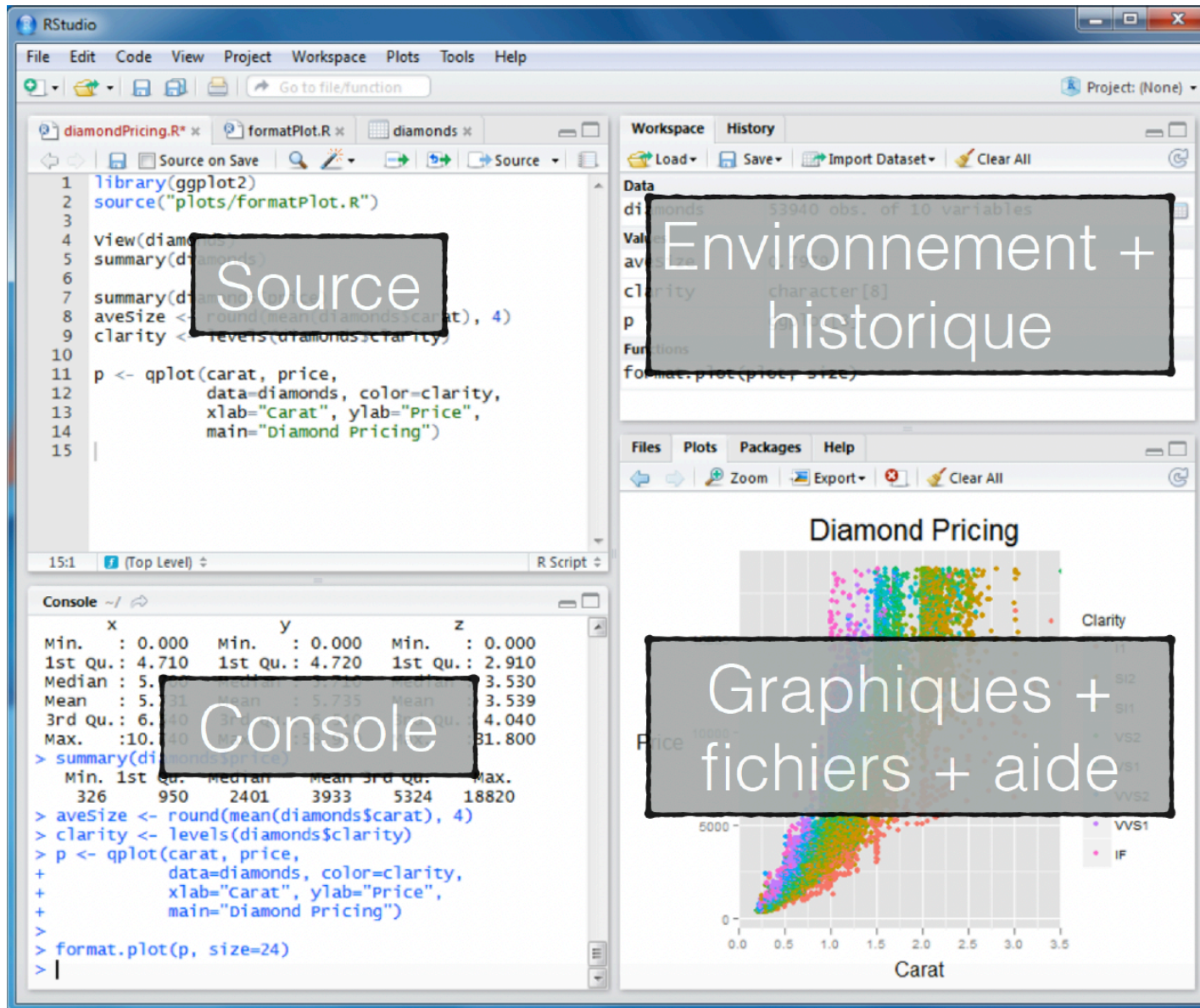
Password:

You will automatically be signed out after 60 minutes of inactivity.

☐ Stay signed in when browser closes

Sign In

RStudio



Environnement de travail pour le développement en R.

- **Source** : vos scripts
- **Console** : vos commandes
- **Environment** : variables, historique, Git, etc.
- **Plots, Packages, Help**

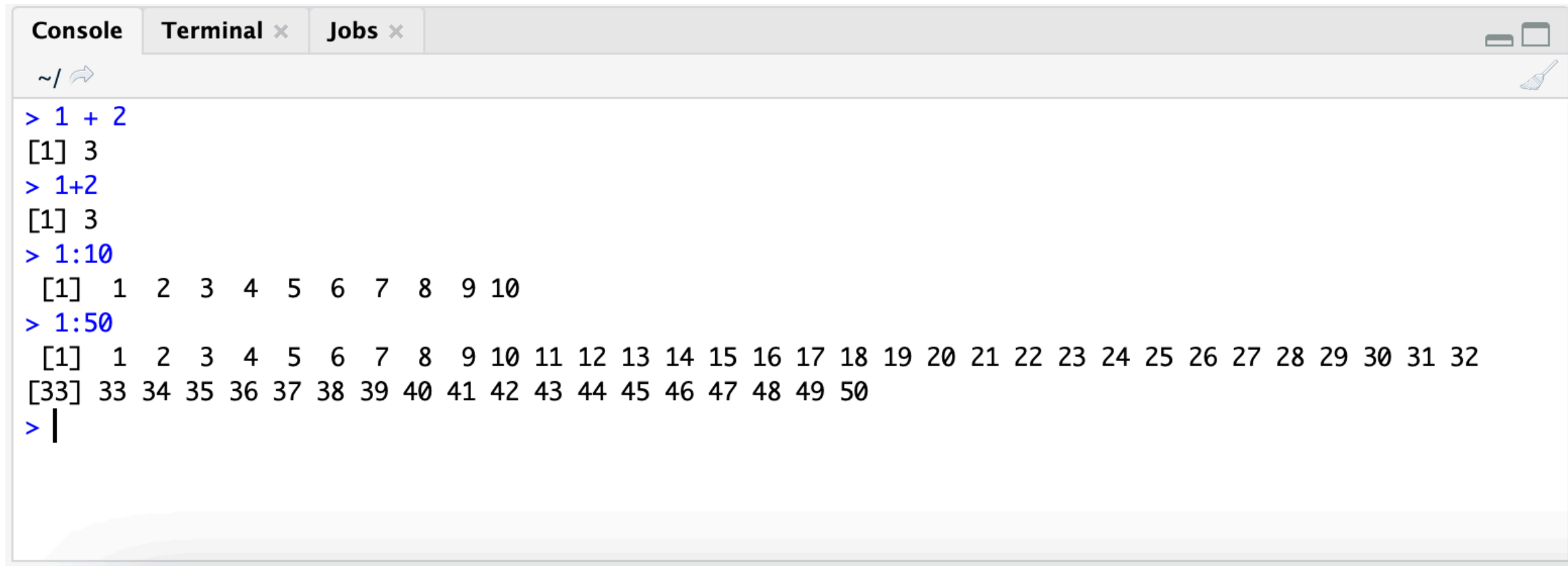
Les commandes écrites dans *Source* peuvent être envoyées à la *Console* (Ctrl+Entrée).

| 3

UN PREMIER CONTACT AVEC R

Premières commandes

Écrivez vos commandes dans la **console** (en bas à gauche), puis appuyez sur **Entrée** pour que R interprète et exécute la commande.



```
Console Terminal x Jobs x
~/
> 1 + 2
[1] 3
> 1+2
[1] 3
> 1:10
[1] 1 2 3 4 5 6 7 8 9 10
> 1:50
[1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32
[33] 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50
> |
```

- **>** est l'invite de commande
- les espaces autour des opérateurs sont ignorés, mais **<-** ne se coupe pas : **x < - 1** compare **x** à -1
- les résultats sont précédés de leur position sur la ligne de retour : **[1]**

Les variables

Les résultats sont seulement affichés dans la console, pas sauvegardés. On utilise des *variables* pour les stocker, avec l'opérateur `<-` :

```
ma_variable <- 1
chien <- "chat"
a <- 1 + 2
```

En tapant le nom de la variable, son contenu s'affiche :

```
ma_variable
#> [1] 1
chien
#> [1] "chat"
a
#> [1] 3
```

Un nom de variable ne commence pas par un chiffre et ne contient pas de caractères spéciaux : `^`, `!`, `$`, `@`, `+`, `-`, `/` ou `*`.

Les vecteurs (1)

R est un langage basé sur la notion de **vecteur** : le calcul est simplifié et nécessite peu de boucles.

On combine des valeurs dans un vecteur avec la fonction **c()** :

```
var1 <- c(1, 2)
var2 <- c("chien", "chat")
var3 <- c(TRUE, TRUE, FALSE)
var1; var2; var3
#> [1] 1 2
#> [1] "chien" "chat"
#> [1] TRUE TRUE FALSE
```

Les opérations s'appliquent à **chaque élément** :

```
var1 * 10
#> [1] 10 20
```

Les vecteurs (2)

On accède aux éléments d'un vecteur par leur position (index), entre crochets **[]** :

```
var <- c("chien", "chat", "mouton", "oiseau")
var[1]
#> [1] "chien"
var[c(2, 4)] # éléments 2 et 4
#> [1] "chat"    "oiseau"
var[2:4]      # éléments 2 à 4 (une tranche)
#> [1] "chat"    "mouton" "oiseau"
```

On peut aussi sélectionner avec un vecteur **logique** :

```
nombres <- c(5, 12, 3, 20)
nombres > 10
#> [1] FALSE  TRUE FALSE  TRUE
nombres[nombres > 10]
#> [1] 12 20
```

👉 À vous : prédire la sortie

Sans exécuter, que renvoie chaque ligne ?

```
x <- c(10, 20, 30, 40)
```

1. `x[2]`

2. `x[2:3]`

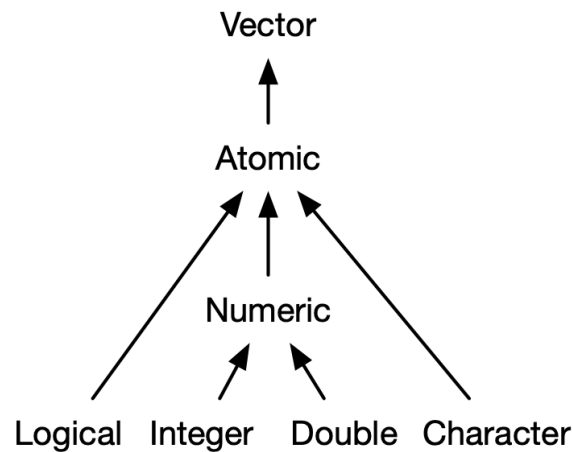
3. `x > 25`

4. `x[x > 25]`

5. `x + 1`

```
x[2]  
#> [1] 20  
x[2:3]  
#> [1] 20 30  
x > 25  
#> [1] FALSE FALSE TRUE TRUE  
x[x > 25]  
#> [1] 30 40  
x + 1  
#> [1] 11 21 31 41
```

Les différents types de scalaires



Advanced R (H. Wickham)

Chaque élément d'un vecteur est un scalaire. Types principaux :

- **numériques** : **double** (3.14 ; -0.65) et **integer** (1L) ;
- **chaînes de caractères** : entre 'simples' ou "doubles" guillemets ;
- **logiques** : **TRUE** / **FALSE**, à écrire en entier (**T** et **F** peuvent être écrasés par une variable).

À part : **NULL** (valeur vide) et **NA** (valeur manquante).

Le type *facteur* (**factor**) représente des données catégorielles :

```
a <- factor(c("chien", "chien", "chat")); a
#> [1] chien chien chat
#> Levels: chat chien
```

👋 À vous : types et valeurs manquantes

Sans exécuter : quel résultat, et pourquoi ?

1. `class(c(1, "a", TRUE))`

2. `class(1 == 1)`

3. `mean(c(2, 4, NA))`

4. `1:6 + c(10, 20)`

```
class(c(1, "a", TRUE)) # tout est converti en texte
#> [1] "character"
class(1 == 1)
#> [1] "logical"
mean(c(2, 4, NA))      # un NA contamine le calcul
#> [1] NA
mean(c(2, 4, NA), na.rm = TRUE)
#> [1] 3
1:6 + c(10, 20)        # c(10, 20) est recyclé
#> [1] 11 22 13 24 15 26
```

Fonctions utiles sur les vecteurs (1)

Créer et résumer un vecteur :

```
seq(1, 10, by = 3)
#> [1] 1 4 7 10
rep("a", 3)
#> [1] "a" "a" "a"
notes <- c(12, 15, 9, 18)
length(notes)
#> [1] 4
```

```
sum(notes)
#> [1] 54
mean(notes)
#> [1] 13.5
max(notes)
#> [1] 18
min(notes)
#> [1] 9
```

Toutes acceptent `na.rm = TRUE` pour ignorer les valeurs manquantes (sauf `seq`, `rep` et `length`).

Fonctions utiles sur les vecteurs (2)

Trier, chercher, compter :

```
q <- c(5, 2, 9)
sort(q)      # les valeurs triées
#> [1] 2 5 9
order(q)     # leurs positions
#> [1] 2 1 3
which(q > 4)
#> [1] 1 3
```

```
couleurs <- c("rouge", "vert", "rouge")
table(couleurs)
#> rouge  vert
#>      2      1
v <- c(3, NA, 7)
is.na(v)
#> [1] FALSE  TRUE FALSE
sum(is.na(v))
#> [1] 1
```

Pour trier les lignes d'un **data.frame**, on utilise les **positions** : **df[order(df\$col),]**, jamais **sort()**.

Les matrices

Structures de dimension 2 dont tous les éléments sont **du même type** :

$$A = \begin{bmatrix} 2 & 4 & 3 \\ 1 & 5 & 7 \end{bmatrix}$$

```
A <- matrix(data = c(2, 1, 4, 5, 3, 7), ncol = 3, nrow = 2, byrow = FALSE)
```

```
A
```

```
#>      [,1] [,2] [,3]
```

```
#> [1,]    2    4    3
```

```
#> [2,]    1    5    7
```

Les matrices (2)

On accède à des lignes, des colonnes, des cases ou des sous-ensembles avec `[ligne, colonne]` :

```
A[2, ]      # la deuxième ligne
#> [1] 1 5 7
A[, 2]      # la deuxième colonne
#> [1] 4 5
A[2, 2]     # la case ligne 2, colonne 2
#> [1] 5
A[, 2:3]    # colonnes 2 à 3, toutes les lignes
#>      [,1] [,2]
#> [1,]    4    3
#> [2,]    5    7
```

| 4

LES data . frame

Les **data.frame**

Tableau à 2 dimensions : un **agrégat de colonnes** (vecteurs de même longueur). Contrairement aux matrices, les colonnes peuvent avoir **des types différents**.

```
col1 <- c(1, 2, 3)
col2 <- c(TRUE, TRUE, FALSE)
col3 <- letters[2:4]
df <- data.frame(col1, col2, col3)
df
#>   col1 col2 col3
#> 1     1 TRUE   b
#> 2     2 TRUE   c
#> 3     3 FALSE  d
```

On accède aux éléments comme dans une matrice, et à une colonne par son nom avec **\$** :

```
df[2, ]
#>   col1 col2 col3
#> 2     2 TRUE   c
df$col3
#> [1] "b" "c" "d"
```

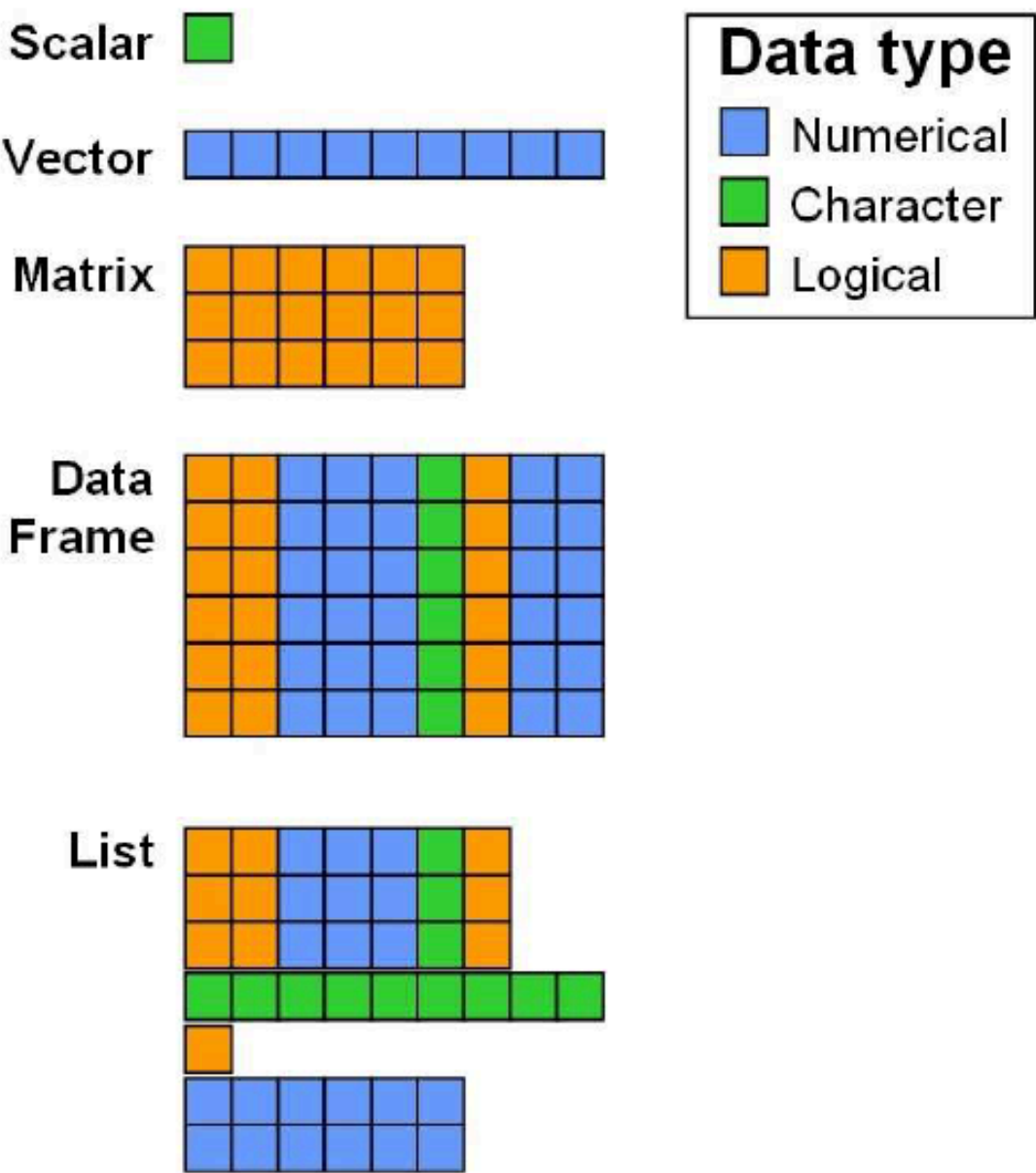
👉 À vous : sélectionner dans un `data.frame`

Avec le `df` de la diapo précédente, que renvoie :

1. `df[3, 1]`
2. `df$col1 > 1`
3. `df[df$col1 > 1, ]`
4. `df[df$col2, "col3"]`

```
df[3, 1]
#> [1] 3
df$col1 > 1
#> [1] FALSE TRUE TRUE
df[df$col1 > 1, ]
#>   col1 col2 col3
#> 2     2  TRUE    c
#> 3     3 FALSE    d
df[df$col2, "col3"]
#> [1] "b" "c"
```

Résumé des structures de base



TP1

| 5

FONCTIONS, AIDE ET PACKAGES

Utiliser une fonction et lire son aide

Une fonction prend des arguments et renvoie un résultat : `nom(argument1, argument2, ...)`

```
x <- c(4, 8, NA, 2)
mean(x)
#> [1] NA
mean(x, na.rm = TRUE)    # argument nommé
#> [1] 4.666667
```

Pour découvrir une fonction : `?mean` ou `help(mean)`. L'aide contient toujours :

- **Usage** : les arguments et leur valeur par défaut (`na.rm = FALSE`)
- **Arguments** : ce que chaque argument attend
- **Value** : ce que la fonction renvoie
- **Exemples** : du code à copier et essayer (le plus utile !)

Lire un message d'erreur

Un message d'erreur n'est pas un échec, c'est une **indication**. Lisez-le avant de chercher ailleurs.

```
moyenne <- mean(c(1, 2))
Moyenne                # R distingue majuscules et minuscules
#> Error: object 'Moyenne' not found
Mean(c(1, 2))           # la fonction n'existe pas sous ce nom
#> Error in Mean(c(1, 2)): could not find function "Mean"
1:10 + c(1, 2, 3)       # un avertissement (warning), pas une erreur
#> [1]  2  4  6  5  7  9  8 10 12 11
```

- **Error** : la commande s'est arrêtée, rien n'a été calculé
- **Warning** : le calcul a été fait, mais le résultat est peut-être faux

Les packages

Un *package* ajoute des fonctions à R. Deux étapes différentes :

```
install.packages("ggplot2") # 1 seule fois : télécharger et installer  
library(ggplot2)           # à chaque session : charger le package
```

- `install.packages()` : une seule fois par ordinateur (ou par compte sur le serveur)
- `library()` : au début de **chaque** script qui utilise le package
- Erreur classique : `could not find function "ggplot"` → le package n'est pas chargé

| 6

LES BOUCLES

Les boucles **for** et la fonction **seq()**

Une boucle répète une action plusieurs fois, souvent sur une séquence créée avec **seq()** :

```
for (variable in sequence) {  
  # instructions à répéter  
}
```

```
for (i in seq(1, 3)) {  
  print(paste("Itération :", i))  
}  
#> [1] "Itération : 1"  
#> [1] "Itération : 2"  
#> [1] "Itération : 3"
```

Avant d'écrire une boucle, vérifiez s'il existe une opération vectorisée : **x * 2** plutôt qu'une boucle qui multiplie chaque élément.

| 7

IMPORTER ET EXPORTER DES DONNÉES

Le répertoire de travail

Un chemin relatif part du répertoire de travail de R, comme dans le terminal Linux :

R	Bash	Rôle
<code>getwd()</code>	<code>pwd</code>	afficher le répertoire de travail
<code>setwd("~/projet")</code>	<code>cd ~/projet</code>	changer de répertoire de travail
<code>list.files()</code>	<code>ls</code>	lister les fichiers du répertoire

```
read.table("donnees.txt")           # relatif : cherché dans getwd()  
read.table("/home/public/R/fruit.tsv") # absolu : fonctionne d'où que l'on soit
```

Dans RStudio : *Session > Set Working Directory*, ou un **projet RStudio** qui fixe le répertoire de travail automatiquement. Évitez les `setwd()` avec un chemin absolu dans un script partagé : il ne fonctionnera que sur votre ordinateur.

Chargement des données

- `read.table()` charge un tableau dans R sous la forme d'un `data.frame`
- `write.table()` sauvegarde un `data.frame` dans un fichier texte
- Dans les 2 cas, il faut le chemin du fichier (chemin relatif ou absolu)

```
var <- read.table("donnees.txt", header = TRUE, sep = "\t")
write.table(var, file = "donnees.txt", sep = "\t", quote = FALSE, row.names = FALSE)
```

Pour un fichier `.CSV` (champs séparés par des virgules) :

```
var <- read.csv("donnees.csv")
write.csv(var, file = "donnees.csv", row.names = FALSE)
```

Écrivez avec les **mêmes réglages** que ceux de la relecture. Par défaut, `write.table()` sépare par des espaces et ajoute guillemets et numéros de rangées, et `write.csv()` ajoute une colonne `X` à la relecture.

Sauvegarder et recharger des objets R :

```
save(var1, var2, file = "donnees.RData") # sauvegarde de var1 et var2
load("donnees.RData")
```

👋 À vous : que s'est-il passé ?

Le fichier `notes.tsv` contient un en-tête et des colonnes séparées par des tabulations :

nom	note
Alice	15
Bob	12

Qu'obtient-on avec chacune de ces commandes ?

1. `read.table("notes.tsv", sep = "\t")`
2. `read.csv("notes.tsv")`

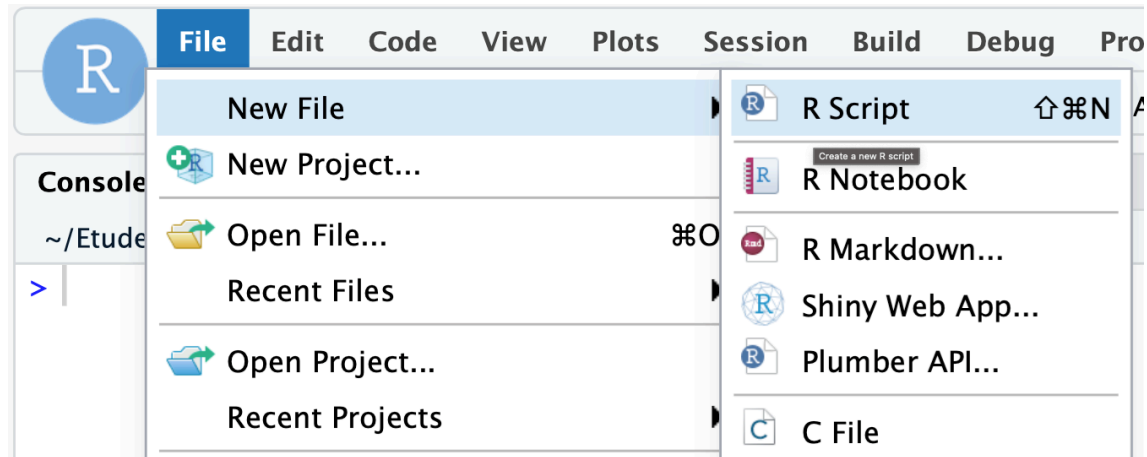
1. 3 lignes : l'en-tête `nom / note` est lu comme une ligne de données (il manque `header = TRUE`), et la colonne des notes devient du texte.
2. Une seule colonne : `read.csv` cherche des virgules (il manque `sep = "\t"`).

Réflexe : après chaque chargement, vérifiez avec `head()`, `dim()` ou le panneau *Environment*.

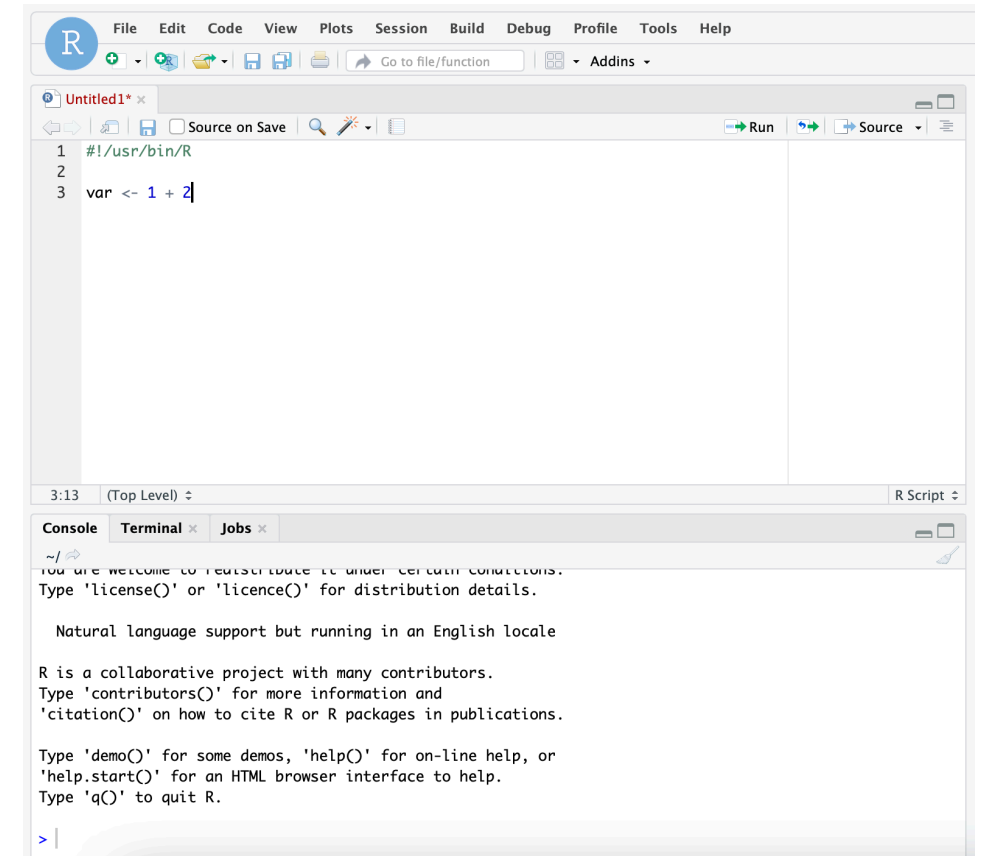
| 8

LES SCRIPTS

Les scripts



- Documente précisément votre travail
- Permet de le reproduire
- Permet de réutiliser votre code pour d'autres projets
- Contient toutes les étapes réalisées (mais pas les erreurs ni les essais)
- Explique en commentaire (#) ce qui n'est pas évident



TP2

TP3

| 9

ACTIVITÉ : AUDITER UN SCRIPT

Auditer un script généré par IA

« Calcule la différence d'expression moyenne de TP53 entre les groupes traité et contrôle, compte les patients dont l'expression dépasse 8, et affiche les 3 patients avec la plus forte expression. »

```
# Analyse de l'expression de TP53 selon le groupe
# Script vérifié et prêt à l'emploi.

donnees <- read.table("/home/public/R/patients.tsv", header = TRUE, sep = "\t")

# 1. Différence d'expression moyenne entre les groupes
moy_traite <- mean(donnees$TP53[donnees$groupe == "traite"], na.rm = TRUE)
moy_controle <- mean(donnees$TP53[donnees$groupe == "controle"], na.rm = TRUE)
moy_traite - moy_controle

# 2. Nombre de patients avec une expression élevée (> 8)
nrow(donnees[donnees$TP53 > 8, ])

# 3. Les 3 patients avec la plus forte expression
donnees[sort(donnees$TP53, decreasing = TRUE)[1:3], ]
```

patients.tsv

id	groupe	age	TP53
P01	traite	34	9.7
P02	controle	51	5.1
P03	traite	47	11.6
P04	Controle	62	4.8
P05	traite	29	NA
P06	controle	55	6.2
P07	traite	41	10.9
P08	Controle	58	3.9
P09	traite	66	8.4
P10	controle	38	5.6
P11	traite	53	9.1
P12	controle	45	7.3

Il s'exécute sans aucune erreur. Ses trois résultats sont pourtant faux.

1. Lire sans exécuter : que fait réellement chaque bloc ? Quelles lignes sont suspectes ?
2. Vérifier dans RStudio : inspecter les données avant de croire les résultats (`head()`, `table()`, `sum(is.na())`)
3. Corriger et commenter chaque correction